

Java Stream API

Complete Learning Guide

30+ Coding Questions with Solutions

Interview Preparation • Beginner to Advanced

Complete Reference Guide 2025

What is Stream API?

Definition

Stream API is a feature introduced in Java 8 that allows functional-style operations on sequences of elements. A Stream represents a sequence of elements and supports different kinds of operations to perform computations on those elements.

Source → Intermediate Operations → Terminal Operation

```
// Stream Pipeline Structure
list.stream() // Source
.filter(x -> x>5) // Intermediate
.map(x -> x*2) // Intermediate
.collect(...) // Terminal
```

Key Characteristics

Characteristic	Description
Not a data structure	Doesn't store data; operates on source (collections, arrays, etc.)
Functional in nature	Operations produce results without modifying source
Lazy evaluation	Computations deferred until terminal operation
Consumable	Can be traversed only once
Parallel-friendly	Easy to parallelize with <code>parallelStream()</code>

```
// Without Stream (Imperative style)
List<String> names = Arrays.asList("alice", "bob", "charlie",
    "david");
List<String> result = new ArrayList<>();
for (String name : names) {
    if (name.length() > 3) {
        result.add(name.toUpperCase());
    }
}

// With Stream (Declarative style)
List<String> result = names.stream()
    .filter(name -> name.length() > 3)
    .map(String::toUpperCase)
    .collect(Collectors.toList());
```

 **Key Point:** Stream API enables declarative programming - you focus on WHAT to do, not HOW to do it.

Creating Streams

Different Ways to Create Streams

1. From Collections

```
List<String> list = Arrays.asList("a", "b", "c");  
Stream<String> stream = list.stream(); // Sequential  
Stream<String> parallelStream = list.parallelStream(); // Parallel
```

2. From Arrays

```
String[] array = {"a", "b", "c"};  
Stream<String> stream1 = Arrays.stream(array);  
Stream<String> stream2 = Arrays.stream(array, 1, 3); // "b", "c"
```

3. From Values

```
Stream<Integer> stream = Stream.of(1, 2, 3, 4, 5);  
Stream<String> empty = Stream.empty();
```

4. From Ranges (Primitive Streams)

```
IntStream.range(1, 5) // 1,2,3,4  
IntStream.rangeClosed(1, 5) // 1,2,3,4,5
```

5. From Files

```
Files.lines(Paths.get("file.txt")).forEach(System.out::println);
```

6. Infinite Streams

```
Stream.generate(Math::random).limit(5).forEach(System.out::println);  
Stream.iterate(0, n -> n+2).limit(5).forEach(System.out::println);
```

 **Note:** Always use limit() with infinite streams to avoid infinite loops!

Intermediate Operations

Filter Operation

Definition: Selects elements matching a condition.

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);

// Filter even numbers
List<Integer> evens = numbers.stream()
    .filter(n -> n % 2 == 0)
    .collect(Collectors.toList());
// Result: [2, 4, 6]

// Multiple conditions
List<Integer> result = numbers.stream()
    .filter(n -> n > 2 && n < 6)
    .collect(Collectors.toList());
// Result: [3, 4, 5]
```

Map Operation

Definition: Transforms each element.

```
List<String> names = Arrays.asList("alice", "bob", "charlie");

// To uppercase
List<String> upper = names.stream()
    .map(String::toUpperCase)
    .collect(Collectors.toList());
// Result: ["ALICE", "BOB", "CHARLIE"]
```

```
List<Integer> lengths = names.stream()
    .map(String::length)
    .collect(Collectors.toList());
// Result: [5, 3, 7]
```

Interview Question Q1: Find Even Numbers

Question 1

Problem: Given a list of integers, find out all the even numbers using Stream functions.

```
List<Integer> list = Arrays.asList(10, 15, 8, 49, 25, 98, 32);

// Solution 1: Using forEach
list.stream()
    .filter(n -> n % 2 == 0)
    .forEach(System.out::println);

// Solution 2: When numbers are given as Array
int[] arr = {10, 15, 8, 49, 25, 98, 32};
Map<Boolean, List<Integer>> result = Arrays.stream(arr)
    .boxed()
    .collect(Collectors.partitioningBy(num -> num % 2 == 0));
System.out.println("Even numbers: " + result.get(true));
```

Output:

```
10
8
98
32
Even numbers: [10, 8, 98, 32]
```

FlatMap Operation

Definition

flatMap() flattens nested structures (one-to-many mapping).

```
// Flatten list of lists
List<List<String>> listOfLists = Arrays.asList(
    Arrays.asList("a", "b"),
    Arrays.asList("c", "d")
);

List<String> flat = listOfLists.stream()
    .flatMap(List::stream)
    .collect(Collectors.toList());
// Result: ["a", "b", "c", "d"]

// Split sentences into words
List<String> sentences = Arrays.asList("Hello World", "Java
Streams");
List<String> words = sentences.stream()
    .flatMap(s -> Arrays.stream(s.split(" ")))
    .collect(Collectors.toList());
// Result: ["Hello", "World", "Java", "Streams"]
```

flatMap vs map

map	flatMap
One-to-one mapping	One-to-many mapping
Returns Stream<R>	Returns Stream<R>

Interview Question Q2: Numbers Starting with 1

Question 2

Problem: Find all numbers starting with 1.

```
List<Integer> myList = Arrays.asList(10, 15, 8, 49, 25, 98, 32);

myList.stream()
    .map(s -> s + "") // Convert integer to String
    .filter(s -> s.startsWith("1"))
    .forEach(System.out::println);

// Alternative with array
int[] arr = {10, 15, 8, 49, 25, 98, 32};
List<String> result = Arrays.stream(arr)
    .boxed()
    .map(s -> s + "")
    .filter(s -> s.startsWith("1"))
    .collect(Collectors.toList());
System.out.println("Numbers starting with 1: " + result);
```

Output:

```
10
15
Numbers starting with 1: [10, 15]
```

Distinct & Sorted Operations

Distinct

Definition: Removes duplicates.

```
List<Integer> numbers = Arrays.asList(1, 2, 2, 3, 3, 3, 4);

List<Integer> unique = numbers.stream()
    .distinct()
    .collect(Collectors.toList());
// Result: [1, 2, 3, 4]
```

Sorted

Definition: Sorts elements.

```
List<Integer> numbers = Arrays.asList(5, 2, 8, 1, 9);

// Natural order
List<Integer> sorted = numbers.stream()
    .sorted()
    .collect(Collectors.toList());
// Result: [1, 2, 5, 8, 9]

// Reverse order
List<Integer> reversed = numbers.stream()
    .sorted(Comparator.reverseOrder())
    .collect(Collectors.toList());
// Result: [9, 8, 5, 2, 1]
```

Question 3

Problem: Find duplicate elements in a list.

```
List<Integer> myList = Arrays.asList(10, 15, 8, 49, 25, 98, 98, 32,
15);

// Find duplicates
Set<Integer> set = new HashSet<>();
myList.stream()
    .filter(n -> !set.add(n))
    .forEach(System.out::println);

// Find unique values
List<Integer> uniqueList = myList.stream()
    .distinct()
    .collect(Collectors.toList());
System.out.println("Unique: " + uniqueList);
```

Output:

```
98
15
Unique: [10, 15, 8, 49, 25, 98, 32]
```

Limit & Skip Operations

Limit

Definition: Truncates a stream to a maximum number of elements.

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

// First 5 elements
List<Integer> first5 = numbers.stream()
    .limit(5)
    .collect(Collectors.toList());
// Result: [1, 2, 3, 4, 5]
```

Skip

Definition: Discards the first n elements of a stream.

```
// Skip first 3 elements
List<Integer> skipped = numbers.stream()
    .skip(3)
    .collect(Collectors.toList());
// Result: [4, 5, 6, 7, 8, 9, 10]
```

Pagination Pattern

```
int page = 2, size = 3;
List<Integer> page2 = numbers.stream()
```

```
.collect(Collectors.toList());  
// Result: [4, 5, 6]
```

Interview Question Q4: Find First Element

Question 4

Problem: Find the first element of the list.

```
List<Integer> myList = Arrays.asList(10, 15, 8, 49, 25, 98, 98, 32,  
15);  
  
myList.stream()  
    .findFirst()  
    .ifPresent(System.out::println);  
  
// With array  
int[] arr = {10, 15, 8, 49, 25, 98, 98, 32, 15};  
Arrays.stream(arr)  
    .boxed()  
    .findFirst()  
    .ifPresent(System.out::print);
```

Output:

```
10  
10
```

Terminal Operations

forEach

Definition: Performs action for each element.

```
list.stream().forEach(System.out::println);  
list.parallelStream().forEach(System.out::println); // Order not  
guaranteed
```

collect (Most Important)

Definition: Collects stream elements into a container.

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");  
  
// To List  
List<String> list = names.stream().collect(Collectors.toList());  
  
// To Set  
Set<String> set = names.stream().collect(Collectors.toSet());  
  
// To Map  
Map<String, Integer> map = names.stream()  
    .collect(Collectors.toMap(  
        Function.identity(),  
        String::length  
    ));  
  
// Joining strings  
String joined = names.stream()
```

```
.collect(Collectors.joining(", "));  
// Result: "Alice, Bob, Charlie"
```

Interview Question Q5: Total Number of Elements

Question 5

Problem: Find the total number of elements in the list.

```
List<Integer> myList = Arrays.asList(10, 15, 8, 49, 25, 98, 98, 32,  
15);  
  
long count = myList.stream()  
.count();  
System.out.println("Total elements: " + count);  
  
// With array  
int[] arr = {10, 15, 8, 49, 25, 98, 98, 32, 15};  
long arrayCount = Arrays.stream(arr)  
.boxed()  
.count();  
System.out.println("Total elements in array: " + arrayCount);
```

Output:

```
Total elements: 9  
Total elements in array: 9
```

Reduce Operation

reduce

Definition: Combines elements into a single result.

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);

// Sum
int sum = numbers.stream()
    .reduce(0, (a, b) -> a + b); // 15

// Max
Optional<Integer> max = numbers.stream()
    .reduce(Integer::max); // Optional[5]

// Product
int product = numbers.stream()
    .reduce(1, (a, b) -> a * b); // 120
```

Interview Question Q6: Find Maximum Value

Question 6

Problem: Find the maximum value element in the list.

```
List<Integer> myList = Arrays.asList(10, 15, 8, 49, 25, 98, 98, 32,
    15);

// Using max() with Integer.compare
int max = myList.stream()
    .max(Integer::compare)
```



```
// Using max() with Comparator
int max2 = myList.stream()
    .max(Comparator.naturalOrder())
    .get();
System.out.println("Maximum value: " + max2);

// With array
int[] arr = {10, 15, 8, 49, 25, 98, 98, 32, 15};
int maxArray = Arrays.stream(arr)
    .boxed()
    .max(Comparator.naturalOrder())
    .get();
System.out.println("Maximum from array: " + maxArray);
```

Output:

```
Maximum value: 98
Maximum value: 98
Maximum from array: 98
```

Match Operations

anyMatch / allMatch / noneMatch

Definition: Check conditions against elements.

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);

// Any even?
boolean anyEven = numbers.stream()
    .anyMatch(n -> n % 2 == 0); // true

// All positive?
boolean allPositive = numbers.stream()
    .allMatch(n -> n > 0); // true

// None negative?
boolean noneNegative = numbers.stream()
    .noneMatch(n -> n < 0); // true
```

Interview Question Q11: Contains Duplicate

Question 11

Problem: Return true if any value appears at least twice in the array.

```
public static boolean containsDuplicate(int[] nums) {
    Set<Integer> setData = new HashSet<>();
    return Arrays.stream(nums)
        .anyMatch(num -> !setData.add(num));
}
```

```
int[] nums2 = {1, 2, 3, 4};  
System.out.println("Array [1,2,3,1] contains duplicate: " +  
containsDuplicate(nums1));  
System.out.println("Array [1,2,3,4] contains duplicate: " +  
containsDuplicate(nums2));
```

Output:

```
Array [1,2,3,1] contains duplicate: true  
Array [1,2,3,4] contains duplicate: false
```

Find Operations

findFirst / findAny

Definition: Find an element (returns Optional).

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);

// findFirst - respects order
Optional<Integer> first = numbers.stream()
    .filter(n -> n > 3)
    .findFirst(); // 4

// findAny - better for parallel streams
Optional<Integer> any = numbers.parallelStream()
    .filter(n -> n > 3)
    .findAny(); // Any matching element

// Safe handling
Integer result = numbers.stream()
    .filter(n -> n > 10)
    .findFirst()
    .orElse(-1); // -1 if not found
```

Interview Question Q7: First Non-Repeated Character

Question 7

Problem: Find the first non-repeated character in a String.

```
String input = "Java articles are Awesome";
```

```

.mapToObj(s -> Character.toLowerCase(Character.valueOf((char) s)))
.collect(Collectors.groupingBy(Function.identity(),
LinkedHashMap::new, Collectors.counting()))
.entrySet()
.stream()
.filter(entry -> entry.getValue() == 1L)
.map(entry -> entry.getKey())
.findFirst()
.get();
System.out.println("First non-repeated character: " + result);

// Alternative using indexOf and lastIndexOf
Character result2 = input.chars()
.mapToObj(c -> (char) c)
.filter(ch -> input.indexOf(ch) == input.lastIndexOf(ch))
.findFirst()
.orElse(null);
System.out.println("First non-repeated character: " + result2);

```

Output:

```

First non-repeated character: j
First non-repeated character: J

```

Collectors - groupingBy

groupingBy (Very Important)

Definition: Groups elements by a classifier function.

```
class Employee {
    String name;
    String department;
    double salary;
    // constructor, getters
}

List<Employee> employees = getEmployees();

// Group by department
Map<String, List<Employee>> byDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment));

// Count by department
Map<String, Long> countByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.counting()
    ));

// Average salary by department
Map<String, Double> avgSalary = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.averagingDouble(Employee::getSalary)
    ));
```

Interview Question Q17: List to Map with Duplicate Keys

Question 17

Problem: Convert a List of objects into a Map handling duplicate keys.

```
class Notes {
    int id;
    String tagName;
    long tagId;
    // constructor, getters
}

List<Notes> noteLst = new ArrayList<>();
noteLst.add(new Notes(1, "note1", 11));
noteLst.add(new Notes(2, "note2", 22));
noteLst.add(new Notes(3, "note3", 33));
noteLst.add(new Notes(4, "note4", 44));
noteLst.add(new Notes(5, "note5", 55));
noteLst.add(new Notes(6, "note4", 66)); // Duplicate tagName

// Convert to Map with duplicate key handling and sorting
Map<String, Long> notesRecords = noteLst.stream()
    .sorted(Comparator.comparingLong(Notes::getTagId).reversed())
    .collect(Collectors.toMap(
        Notes::getTagName,
        Notes::getTagId,
        (oldValue, newValue) -> oldValue, // Keep old value for duplicates
        LinkedHashMap::new
    ));

System.out.println("Notes Map: " + notesRecords);
```

Output:

Notes Map: {note5=55, note4=44, note3=33, note2=22, note1=11}

Collectors - partitioningBy

partitioningBy

Definition: Partitions elements into two groups based on a predicate.

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);

// Partition into even and odd
Map<Boolean, List<Integer>> partitioned = numbers.stream()
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));

List<Integer> evens = partitioned.get(true); // [2, 4, 6]
List<Integer> odds = partitioned.get(false); // [1, 3, 5]

// Partition with count
Map<Boolean, Long> countByParity = numbers.stream()
    .collect(Collectors.partitioningBy(
        n -> n % 2 == 0,
        Collectors.counting()
    ));
// {false=3, true=3}
```

Interview Question Q18: Count Each Element

Question 18

Problem: Count each element/word from the String ArrayList.

```
List<String> names = Arrays.asList("AA", "BB", "AA", "CC");

// Using groupingBy with counting
```



```
Function.identity(),
Collectors.counting()
));
System.out.println("Element counts: " + namesCount);

// Count words in a sentence
String sentence = "Java is great and Java is powerful";
List<String> words = Arrays.asList(sentence.split(" "));
Map<String, Long> wordCount = words.stream()
    .collect(Collectors.groupingBy(
        Function.identity(),
        Collectors.counting()
    ));
System.out.println("Word counts: " + wordCount);
```

Output:

Element counts: {CC=1, BB=1, AA=2}

Word counts: {Java=2, great=1, is=2, and=1, powerful=1}

Primitive Streams

When to Use

Use when working with large numbers of primitives to avoid boxing overhead.

Creating Primitive Streams

```
// From ranges
IntStream.range(1, 10);
IntStream.rangeClosed(1, 10);

// From values
IntStream.of(1, 2, 3, 4, 5);

// From arrays
int[] array = {1, 2, 3, 4, 5};
IntStream stream = Arrays.stream(array);
```

Important Methods

```
IntStream stream = IntStream.rangeClosed(1, 10);

int sum = stream.sum(); // 55
double avg = stream.average().orElse(0); // 5.5
int min = stream.min().orElse(0); // 1
int max = stream.max().orElse(0); // 10

// All statistics at once
IntSummaryStatistics stats = stream.summaryStatistics();
stats.getCount();
stats.getSum();
```

```
stats.getMax();  
stats.getAverage();
```

Interview Question Q21: Find Maximum Element in Array

Question 21

Problem: Find the Maximum element in an array.

```
int[] arr = {12, 19, 20, 88, 00, 9};  
  
// Using max() with getAsInt()  
int max1 = Arrays.stream(arr)  
    .max()  
    .getAsInt();  
System.out.println("Maximum element: " + max1);  
  
// Using reduce  
int max2 = Arrays.stream(arr)  
    .reduce(Integer.MIN_VALUE, (a, b) -> a > b ? a : b);  
System.out.println("Maximum element: " + max2);  
  
// Using summaryStatistics  
IntSummaryStatistics stats = Arrays.stream(arr)  
    .summaryStatistics();  
System.out.println("Maximum element: " + stats.getMax());
```

Output:

```
Maximum element: 88  
Maximum element: 88  
Maximum element: 88
```

Additional Coding Questions

Question 8: First Repeated Character

Question 8

```
String input = "Java Articles are Awesome";

// Using Set
Set<Character> seenCharacters = new HashSet<>();
Character result = input.chars()
    .mapToObj(c -> (char) c)
    .filter(c -> !seenCharacters.add(c))
    .findFirst()
    .orElse(null);
System.out.println("First repeated character: " + result);
```

Output:

First repeated character: a

Question 9: Sort Values (Ascending)

Question 9

```
List<Integer> myList = Arrays.asList(10, 15, 8, 49, 25, 98, 98, 32,
15);

List<Integer> sortedList = myList.stream()
    .sorted()
    .collect(Collectors.toList());
```

Output:

Sorted list: [8, 10, 15, 15, 25, 32, 49, 98, 98]

Question 10: Sort Values (Descending)

Question 10

```
List<Integer> myList = Arrays.asList(10, 15, 8, 49, 25, 98, 98, 32, 15);

List<Integer> sortedDesc = myList.stream()
    .sorted(Comparator.reverseOrder())
    .collect(Collectors.toList());

System.out.println("Sorted descending: " + sortedDesc);
```

Output:

Sorted descending: [98, 98, 49, 32, 25, 15, 15, 10, 8]

More Coding Questions

Question 12: Current Date and Time

Question 12

```
System.out.println("Current Local Date: " + LocalDate.now());
System.out.println("Current Local Time: " + LocalTime.now());
System.out.println("Current Local Date and Time: " +
LocalDateTime.now());
```

Question 13: Concatenate Two Streams

Question 13

```
List<String> list1 = Arrays.asList("Java", "8");
List<String> list2 = Arrays.asList("explained", "through",
"programs");

Stream<String> concatStream = Stream.concat(list1.stream(),
list2.stream());
concatStream.forEach(str -> System.out.print(str + " "));
```

Output:

Java 8 explained through programs

Question 14: Cube and Filter

```
List<Integer> integerList = Arrays.asList(4, 5, 6, 7, 1, 2, 3);

List<Integer> result = integerList.stream()
    .map(i -> i * i * i)
    .filter(i -> i > 50)
    .collect(Collectors.toList());
System.out.println("Cubes greater than 50: " + result);
```

Output:

Cubes greater than 50: [64, 125, 216, 343]

Question 15: Sort Array and Convert to Stream

Question 15

```
int arr[] = {99, 55, 203, 99, 4, 91};

Arrays.parallelSort(arr);
System.out.println("Sorted array: " + Arrays.toString(arr));
```

Output:

Sorted array: [4, 55, 91, 99, 99, 203]

Advanced Coding Questions

Question 16: Convert to Uppercase

Question 16

```
List<String> names = Arrays.asList("aa", "bb", "cc", "dd");

List<String> upper = names.stream()
    .map(String::toUpperCase)
    .collect(Collectors.toList());

System.out.println("Uppercase: " + upper);
```

Output:

Uppercase: [AA, BB, CC, DD]

Question 19: Find Duplicate Elements with Count

Question 19

```
List<String> names = Arrays.asList("AA", "BB", "AA", "CC", "BB",
    "DD", "AA");

Map<String, Long> duplicates = names.stream()
    .collect(Collectors.groupingBy(Function.identity(),
    Collectors.counting()))
    .entrySet()
    .stream()
    .filter(entry -> entry.getValue() > 1)
    .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue));
```


Output:

Duplicates with count: {AA=3, BB=2}

Question 20: Check if List is Empty

Question 20

```
List<String> noteLst = Arrays.asList("note1", "note2", "note3");

Optional.ofNullable(noteLst)
    .orElseGet(Collections::emptyList)
    .stream()
    .filter(Objects::nonNull)
    .forEach(System.out::println);
```

Output:

note1
note2
note3

Question 22: Count Each Character in String

Question 22

```
String s = "string data to count each character";

Map<String, Long> charCount = Arrays.stream(s.split(""))
    .map(String::toLowerCase)
    .collect(Collectors.groupingBy(
        Function.identity(),
        Collectors.counting()
    ));

System.out.println("Character counts: " + charCount);
```

Additional Advanced Questions

Question 23: Find Second Highest Number

Question 23

```
List<Integer> numbers = Arrays.asList(5, 2, 8, 1, 9, 3, 7, 4, 6);

Optional<Integer> secondHighest = numbers.stream()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst();
System.out.println("Second highest: " + secondHighest.orElse(-1));
```

Output:

Second highest: 8

Question 24: Sum of Squares of Even Numbers

Question 24

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);

int sum = numbers.stream()
    .filter(n -> n % 2 == 0)
    .mapToInt(n -> n * n)
    .sum();
System.out.println("Sum of squares of even numbers: " + sum);
```

Output:

Sum of squares of even numbers: 56

Question 25: Find Longest String

Question 25

```
List<String> words = Arrays.asList("Java", "Python", "C++",  
    "JavaScript", "Go");  
  
Optional<String> longest = words.stream()  
    .max(Comparator.comparing(String::length));  
System.out.println("Longest string: " + longest.orElse(""));
```

Output:

Longest string: JavaScript

Question 26: Partition Numbers into Even and Odd

Question 26

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);  
  
Map<Boolean, List<Integer>> partitioned = numbers.stream()  
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));  
System.out.println("Even numbers: " + partitioned.get(true));  
System.out.println("Odd numbers: " + partitioned.get(false));
```

Output:

Even numbers: [2, 4, 6, 8, 10]
Odd numbers: [1, 3, 5, 7, 9]

Most Asked Interview Questions

Q1: What is the difference between Collection and Stream?

Feature	Collection	Stream
Storage	Stores data	Doesn't store data
Modification	Can add/remove	Cannot modify
Traversal	Multiple times	Only once
Evaluation	Eager	Lazy

Q2: What are intermediate and terminal operations?

Intermediate Operations: Return a new stream, are lazy, can be chained.

Examples: filter(), map(), sorted(), distinct(), limit(), skip()

Terminal Operations: Produce result, eager, consume the stream.

Examples: collect(), forEach(), count(), reduce(), anyMatch(), findFirst()

Q3: What is the difference between map and flatMap?

map	flatMap
One-to-one mapping	One-to-many mapping
Returns Stream<R>	Returns Stream<R>

Q4: What is lazy evaluation in Streams?

Intermediate operations are not executed until a terminal operation is invoked. This improves performance, allows short-circuiting, and saves memory.

Q5: Can we reuse a stream after terminal operation?

No. Stream is consumed after terminal operation.

```
Stream<String> stream = list.stream();  
stream.forEach(System.out::println);  
stream.forEach(System.out::println); // Throws IllegalStateException
```

More Interview Questions

Q6: What is the difference between `findFirst` and `findAny`?

findFirst: Returns first element (respects encounter order)

findAny: Returns any element (better for parallel streams)

Q7: What is short-circuiting in Streams?

Operations that don't need to process entire stream to produce result.

Examples: `limit()`, `findFirst()`, `findAny()`, `anyMatch()`, `allMatch()`, `noneMatch()`

Q8: What is the difference between `parallelStream` and `stream`?

stream(): Sequential processing (single thread)

parallelStream(): Parallel processing (multiple threads)

Q9: What is `Optional` and why is it used in Streams?

`Optional` is a container object that may or may not contain a value. It's used to avoid `NullPointerException`.

Q10: What are the advantages of using Stream API?

- Concise code - Less boilerplate
- Readability - Pipeline reads like business logic
- Parallelism - Easy to parallelize
- Lazy evaluation - Better performance
- Functional programming - Declarative style
- No side effects - Operations don't modify source

Quick Reference Cheat Sheet

Intermediate Operations

Operation	What it does
<code>filter(predicate)</code>	Keeps elements that match condition
<code>map(function)</code>	Transforms each element
<code>flatMap(function)</code>	Flattens nested structures
<code>distinct()</code>	Removes duplicates
<code>sorted()</code>	Sorts elements
<code>limit(n)</code>	Takes first n elements
<code>skip(n)</code>	Skips first n elements
<code>peek(action)</code>	Debugging - performs action

Terminal Operations

Operation	What it returns
<code>collect(collector)</code>	Collection, Map, String
<code>forEach(action)</code>	void
<code>reduce(identity, accumulator)</code>	value

anyMatch(predicate)	boolean
allMatch(predicate)	boolean
noneMatch(predicate)	boolean
findFirst()	Optional
findAny()	Optional
min(comparator)	Optional
max(comparator)	Optional

Common Collectors

Collector	What it does
toList()	Collects to List
toSet()	Collects to Set
toMap(keyMapper, valueMapper)	Collects to Map
joining(delimiter)	Joins strings
groupingBy(classifier)	Groups elements
partitioningBy(predicate)	Partitions by condition
counting()	Counts elements
summingInt(mapper)	Sums integer values
averagingInt(mapper)	Averages integer values
summarizingInt(mapper)	Gets all statistics

Java Stream API Summary



Congratulations! You've mastered Java Stream API! 🎉

Key Points to Remember:

- Streams are for processing data, not storing it
- Operations are lazy - only execute at terminal operation
- Streams can be used only once
- Use `parallelStream()` for large datasets
- Collectors are powerful for collecting results
- `Optional` prevents `NullPointerException`
- Practice the 30+ coding questions for interviews

Most Common Stream Patterns:

```
// Filter + Map + Collect (Most Common)
list.stream()
    .filter(condition)
    .map(transformation)
    .collect(Collectors.toList());

// Grouping
list.stream()
    .collect(Collectors.groupingBy(classifier));
```

```
.filter(condition)
.findFirst()
.orElse(defaultValue);
```

Happy Coding with Java Streams! 🚀

30+ Coding Questions • Complete Reference • Interview Ready